

ARCADE

Roadmap du projet — C++ / Epitech

Vue d'ensemble

Le projet Arcade est une plateforme de jeux rétro en C++. L'architecture repose entièrement sur des bibliothèques dynamiques (.so) chargées à l'exécution via `dlopen/dlclose/dlsym`. Le core ne connaît aucune bibliothèque graphique ou jeu directement.

Stack technique

- Langage : C++17
- Build : CMake (out-of-source build dans `./build/`)
- Bibliothèques graphiques : nCurses, SDL2, SFML
- Jeux : Snake, Nibbler
- Flag obligatoire : `-fno-gnu-unique`

Structure du dépôt

Dossier / Fichier	Rôle
<code>src/core/</code>	Core : chargement des libs, boucle principale, menu
<code>src/graphics/ncurses/</code>	Implémentation nCurses
<code>src/graphics/sdl2/</code>	Implémentation SDL2
<code>src/graphics/sfml/</code>	Implémentation SFML
<code>src/games/snake/</code>	Jeu Snake
<code>src/games/nibbler/</code>	Jeu Nibbler
<code>include/</code>	Interfaces partagées : <code>IGame</code> , <code>IDisplay</code>
<code>lib/</code>	Sortie des .so générés par CMake
<code>doc/</code>	Documentation obligatoire (PDF)
<code>CMakeLists.txt</code>	Fichier de build à la racine

Phases de développement

Phase 1 — Interfaces & Architecture (Priorité absolue)

Avant toute implémentation, définir les interfaces C++ pures que toutes les libs devront respecter. C'est le contrat partagé entre le core et les plugins.

Tâche	Priorité	Notes
Définir <code>IDisplay</code> (interface graphique)	CRITIQUE	Pure abstract class, sans

		dépendance SDL/SFML/ncurses
Définir IGame (interface jeu)	CRITIQUE	Pure abstract class, sans logique de rendu
Définir les types partagés (Event, Sprite...)	CRITIQUE	Header-only dans include/
Partager les interfaces avec un autre groupe	Obligatoire	Requis par le sujet

Phase 2 — Core & chargement dynamique

Le coeur du programme : charger les .so, instancier les libs via dlopen, gérer les switches à la volée.

Tâche	Priorité	Notes
Encapsulation de dlopen/dlclose/dlsym	CRITIQUE	Classe DLLoader, pas d'appel direct dans le core
Chargement des .so depuis ./lib/	CRITIQUE	Scan du dossier au démarrage
Vérification : lib graphique vs lib jeu	Obligatoire	Exit 84 si mauvaise lib passée en arg
Switch de lib graphique à la volée (touche)	Obligatoire	Sans quitter le programme
Switch de jeu à la volée (touche)	Obligatoire	Sans quitter le programme
Menu : affichage libs dispo + scores + nom	Obligatoire	Boxes séparées
Gestion des scores	Obligatoire	Registre persistant

Phase 3 — Bibliothèques graphiques

Implémenter les 3 libs graphiques. Commencer par nCurses (la plus simple), puis SFML, puis SDL2.

Tâche	Priorité	Notes
arcade_ncurses.so	Facile	Terminal pur, commencer par celle-ci
arcade_sfml.so	Moyen	API haut niveau, bien documentée
arcade_sdl2.so	Moyen	Plus bas niveau que SFML

Phase 4 — Jeux

Implémenter Snake en premier. Nibbler réutilise une grande partie de la logique Snake — c'est l'avantage de ce choix.

Tâche	Priorité	Notes
arcade_snake.so	Facile	Commencer par celui-ci, logique de base

arcade_nibbler.so	Facile+	Réutilise la logique Snake + ajout du labyrinthe
-------------------	---------	--

Phase 5 — Documentation

Tâche	Priorité	Notes
Doc d'intégration (comment ajouter une lib)	Obligatoire	PDF dans ./doc/
Diagramme de classes	Bonus	Très apprécié
README.md avec email groupe partenaire	Obligatoire	Requis par le sujet

Build CMake

Workflow de compilation :

- `mkdir ./build/ && cd ./build/`
- `cmake .. -G "Unix Makefiles" -DCMAKE_BUILD_TYPE=Release`
- `cmake --build .`
- `cd .. && ls ./arcade ./lib/`

Résultat attendu après build :

- `./arcade` (exécutable à la racine)
- `./lib/arcade_ncurses.so`
- `./lib/arcade_sdl2.so`
- `./lib/arcade_sfml.so`
- `./lib/arcade_snake.so`
- `./lib/arcade_nibbler.so`

Règles critiques à ne pas oublier

- **Idd ./arcade ne doit PAS montrer de dépendance vers les .so de jeux/graphiques**
- **Flag -fno-gnu-unique obligatoire sur toutes les cibles**
- **Le core ne doit JAMAIS inclure un header SDL2/SFML/ncurses directement**
- **Les libs de jeu ne doivent contenir aucune logique de rendu**
- **Les .so doivent être nommés exactement comme dans le sujet (arcade_*.so)**
- **Exit code 84 pour toute erreur, 0 si succès**